Проект по курсу "Базы данных"

Приложение "Кошелёк"

Выполнили студентки группы БПИ203: Арунова Анастасия Дмитриевна, Тей Даяна Игоревна, Бармина Алиса Дмитриевна

Описание

Проект предоставляет пользователю возможность создавать кошельки для отслеживания своих доходов и расходов.

Сущности

В проекте есть следующие сущности:

```
- Пользователь
- Валюта
- Кошелёк
- Категория (дохода/расхода)
- Операция
```

Пользователь

Пользователь должен иметь возможность регистрации/авторизации при помощи ***почты*** и ***пароля***.

Валюта

Валюта должна иметь ***код*** в виде трёх заглавных латинских букв и должна иметь своё ***название***.

Кошелёк

Пользователь должен иметь возможность создавать кошельки. Для каждого кошелька определяется ***валюта***, в которой он создан, ***название*** кошелька, ***лимит*** (максимальная сумма, которая может быть в кошельке), текущий ***баланс***.

Категория

Категория - характеристика операции над кошельком. Она должна обладать ***типом*** - доход (`INCOME`) или расход (`EXPENSE`), и должна иметь своё ***название***. У пользователя должна быть возможность создавать свои собственный категории.

Операция

Операции производятся над кошельком. Каждая операция характеризуется ***категорией***, ***датой и временем***, когда она произошла, и ***суммой***, которая была добавлена/снята с баланса кошелька.

Функциональные требования

Пользователь

- **Регистрация пользователя**

  При успешной регистрации в БД должна появляться запись о пользователе.

- **Удаление пользователя**

  Если пользователь захотел удалить свой аккаунт, необходимо удалять его из БД.

Валюты

- **Добавление валюты/валют**

  Данная функция необходима для добавления списка валют. Доступ к добавлению должен иметь только администратор, для пользователей приложением она должна быть недоступна.

- **Получение списка всех валют**

  Функция нужна для предоставления пользователю списка валют при выборе валюты для кошелька на этапе его создания.

Кошелёк

- **Создание кошелька**

  Каждый пользователь должен иметь возможность создать кошелёк в определённой валюте, с текущим балансом, лимитом на максимальную сумму и названием кошелька.

- **Получение всех кошельков пользователя**

  Пользователю должны отображаться все созданные им кошельки.

- **Удаление кошелька**

  Пользователь должен иметь возможность удалить созданный им кошелёк.

Категория

- **Добавление категорий по умолчанию**

  Данная функция необходима для создания списка категорий по умолчанию, т.е. таких категорий, которые будут отображаться у всех пользователей. Доступ к добавлению категорий по умолчанию должен иметь только администратор.

- **Добавление пользовательских категорий**

  Каждый пользователь должен иметь возможность создать собственную категорию.

- **Получение всех доступных пользователю категорий**

  В данной функции надо получать все доступные пользователю категории, т.е. созданные им категории и категории по умолчанию. Функция нужна при создании операции, когда необходимо выбирать её категорию.

- **Получение категорий пользователя по подстроке в названии**

  Категорий может быть много, поэтому надо предоставлять пользователю их поиск. Он будет осуществляться по вхождению введённой строки в любую часть названия.

- **Получение категорий пользователя по типу**

Категорий может быть много, поэтому надо предоставлять пользователю их поиск. Он будет осуществляться по типу: доход/расход.

Операции

- **Добавление операции по кошельку**

  Пользователь должен иметь возможность производить операции над кошельком (класть/расходовать деньги), которые будут изменять его баланс.

- **Удаление операции их кошелька**

  Пользователь должен иметь возможность отменять добавленные операции.

- **Получение общей суммы расходов и доходов по кошельку за весь период.**

  Для анализа операций по кошельку пользователю должна предоставляться возможность получать общую сумму доходов и расходов по кошельку за весь период.

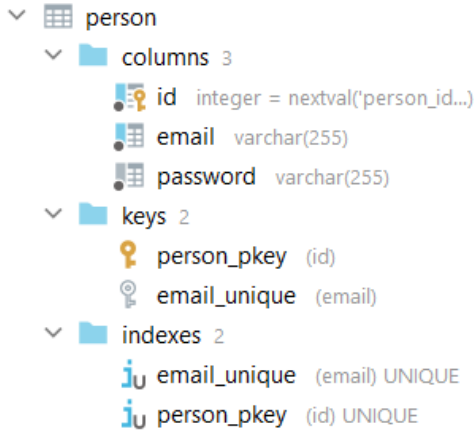
- **Получение общей суммы расходов и доходов по кошельку за последний месяц.**

  Для анализа операций по кошельку пользователю должна предоставляться возможность получать общую сумму доходов и расходов по кошельку за последний месяц.

Архитектура

Пользователь

```
CREATE TABLE person (
    id       serial       PRIMARY KEY,
    email    varchar(255) NOT NULL,
    password varchar(255) NOT NULL,
    CONSTRAINT email_unique UNIQUE (email)
);
```

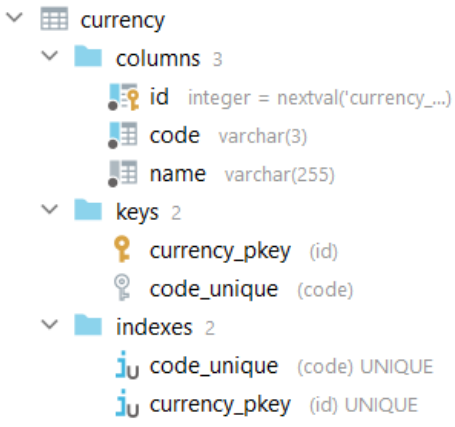


Уникальность:

```
email_unique - e-mail пользователя уникален.
```

Валюта

```
CREATE TABLE currency (
    id   serial       PRIMARY KEY,
    code varchar(3)   NOT NULL,
    name varchar(255) NOT NULL,
    CONSTRAINT code_unique UNIQUE (code)
);
```

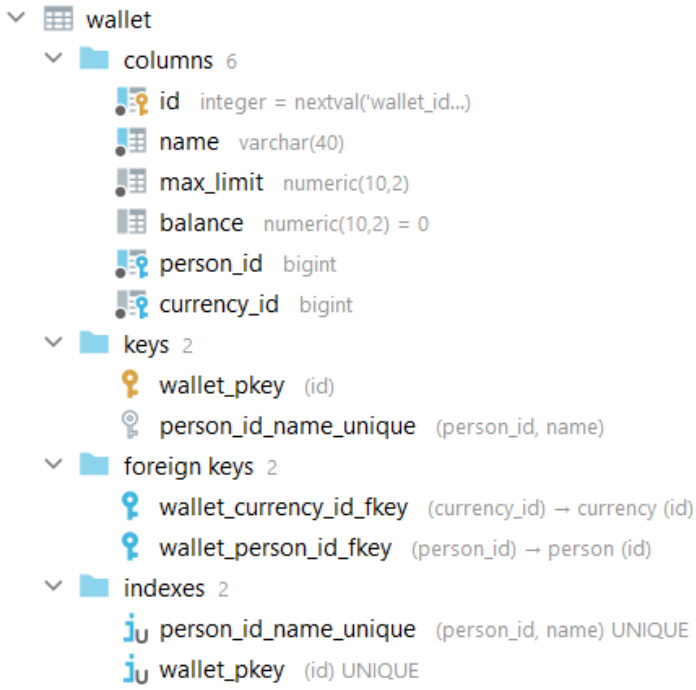


Уникальность:

```
code_unique - код валюты уникален.
```

Кошелёк

```
CREATE TABLE wallet (
    id          serial         PRIMARY KEY,
    name        varchar(40)    NOT NULL,
    max_limit   decimal(10,2) NOT NULL,
    balance     decimal(10,2) DEFAULT 0,
    person_id   bigint         REFERENCES   person   (id) NOT NULL,
    currency_id bigint         REFERENCES   currency (id) NOT NULL,
    CONSTRAINT person_id_name_unique UNIQUE (person_id, name)
);
```



Внешние ключи:

```
person_id - внешний ключ на таблицу person. Каждый кошелёк создаётся
пользователем.
currency_id - внешний ключ на таблицу currency. Каждый кошелёк может быть создан
только в одной валюте.
```

Связи:

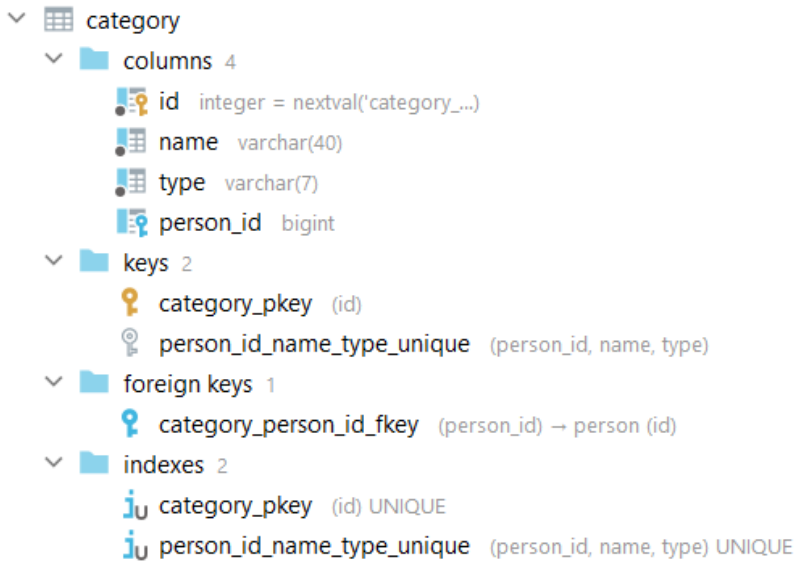
```
wallet-person: many-to-one, однонаправленная, владелец связи wallet.
wallet-currency: many-to-one, однонаправленная, владелец связи wallet.
```

Уникальность:

```
person_id_name_unique - у одного пользователя может быть только один кошелёк с
конкретным названием.
```

Категория

```
CREATE TABLE category (
    id          serial          PRIMARY KEY,
    name        varchar(40)     NOT NULL,
    type        varchar(7)      NOT NULL,
    person_id   bigint          REFERENCES    person (id),
    CONSTRAINT person_id_name_type_unique UNIQUE (person_id, name, type)
);
```



Внешние ключи:

```
person_id - внешний ключ на таблицу person. Категория может быть создана пользователем.
            В табице есть  категории "по умолчанию" для всех пользователей (для них person_id = null).
```

Связи:

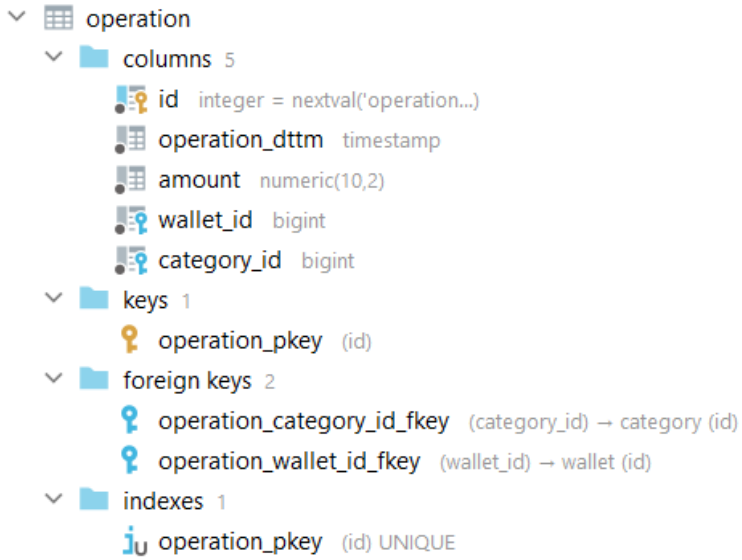
```
category-person: many-to-one, однонаправленная, владелец связи category.
```

Уникальность:

```
person_id_name_type_unique - пользователь может создать только одну категорию с заданным именем и типом.
```

Операция

```
CREATE TABLE operation (
    id             serial        PRIMARY KEY,
    operation_dttm timestamp     NOT NULL,
    amount         decimal(10,2) NOT NULL,
    wallet_id      bigint        REFERENCES   wallet   (id) NOT NULL,
    category_id    bigint        REFERENCES   category (id) NOT NULL
);
```



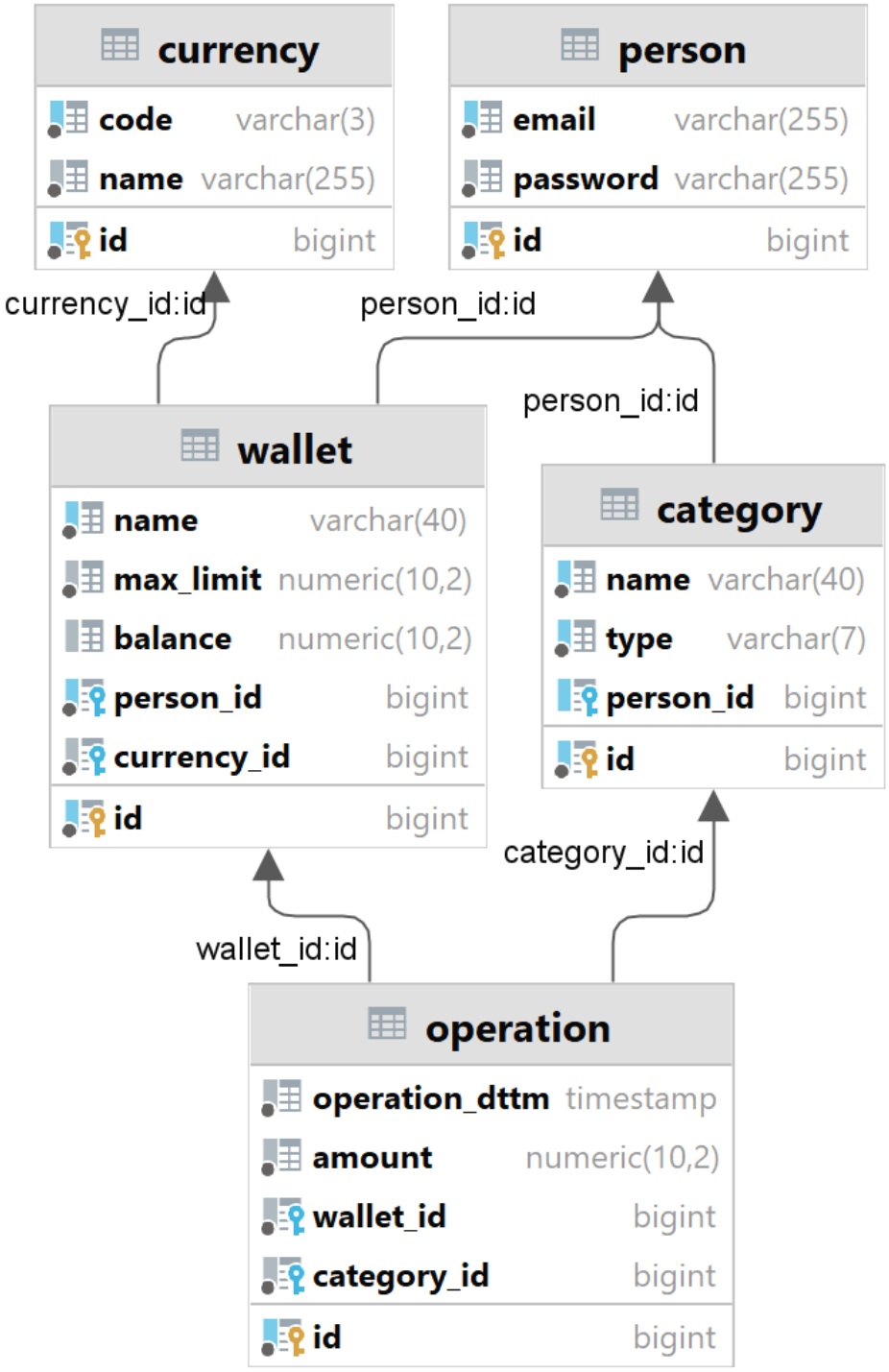
Внешние ключи:

```
wallet_id - внешний ключ на таблицу wallet. Операция производится над конкретным кошельком.
category_id - внешний ключ на таблицу category. Операции соответствует категория.
```

Связи:

```
operation-wallet: many-to-one, однонаправленная, владелец связи operation.
operation-category: many-to-one, однонаправленная, владелец связи operation.
```

UML


currency
code varchar(3)
name varchar(255)
id bigint
person
email varchar(255)
password varchar(255)
id bigint
currency_id:id
person_id:id
person_id:id
wallet
name varchar(40)
max_limit numeric(10,2)
balance numeric(10,2)
person_id bigint
currency_id bigint
id bigint
category
name varchar(40)
type varchar(7)
person_id bigint
id bigint
category_id:id
wallet_id:id
operation
operation_dttm timestamp
amount numeric(10,2)
wallet_id bigint
category_id bigint
id bigint

Скрипты

Добавление пользователя/пользователей

- Заполним таблицу некоторыми пользователями.

```
INSERT INTO person (email, password) VALUES
    ('first@gmail.com', 'password_1'),
    ('second@gmail.com', 'password_2'),
    ('third@gmail.com', 'password_3'),
    ('fourth@gmail.com', 'password_4'),
    ('fifth@gmail.com', 'password_5');
```

	id	email	password
1	1	first@gmail.com	password_1
2	2	second@gmail.com	password_2
3	3	third@gmail.com	password_3
4	4	fourth@gmail.com	password_4
5	5	fifth@gmail.com	password_5

- Скрипт, который выполняется, когда пользователь успешно зарегистрировался:

```
INSERT INTO person (email, password) VALUES (?, ?);
```

Удаление пользователя

```
DELETE FROM operation WHERE wallet_id IN (SELECT id FROM wallet WHERE person_id
= ?);
DELETE FROM wallet WHERE person_id = ?;
DELETE FROM category WHERE person_id = ?;
DELETE FROM person WHERE id = ?;
```

Добавление валюты/валют

Добавлять валюты может только администратор.

```
INSERT INTO currency (code, name) VALUES
    ('RUB', 'Российский рубль'),
    ('USD', 'Доллар США'),
    ('EUR', 'Евро'),
    ('CNY', 'Китайский юань');
```

	id	code	name
1	1	RUB	Российский рубль
2	2	USD	Доллар США
3	3	EUR	Евро
4	4	CNY	Китайский юань

Получение списка всех валют

Данный список необходимо предоставлять пользоввателю при выборе валюты кошелька при его создании. Скрипт для получения списка валют:

```
SELECT * FROM currency;
```

Создание кошелька

- Заполним таблицу какими-нибудь кошельками.

```
INSERT INTO wallet (name, max_limit, person_id, currency_id) VALUES
    ('Кошелёк 1', 1000000, 1, 1), -- Кошелёк в RUB для пользователя с id = 1
    ('Кошелёк 2', 10000, 1, 2); -- Кошелёк в USD для пользователя с id = 1
    -- Баланс на обоих кошельках по умолчанию 0.

INSERT INTO wallet (name, max_limit, balance, person_id, currency_id) VALUES
    ('Кошелёк 3', 10000, 1000, 1, 1), -- Кошелёк в RUB для пользователя с id = 1
    ('Кошелёк 1', 100, 30, 2, 3); -- Кошелёк в EUR для пользователя с id = 2
```

	id	name	max_limit	balance	person_id	currency_id
1	1	Кошелёк 1	1000000.00	0.00	1	1
2	2	Кошелёк 2	10000.00	0.00	1	2
3	3	Кошелёк 3	10000.00	1000.00	1	1
4	4	Кошелёк 1	100.00	30.00	2	3

- Скрипт для создания кошелька конкретным пользователем:

```
-- balance указывать не обязательно.
INSERT INTO wallet (name, max_limit, balance, person_id, currency_id) VALUES
    (?, ?, ?, ?, ?);
```

Получение всех кошельков пользователя

```
SELECT * FROM wallet WHERE wallet.person_id = ?;
```

Удаление кошелька

```
DELETE FROM operation WHERE operation.wallet_id = ?;
DELETE FROM wallet WHERE id = ?;
```

Добавление категорий по умолчанию

Добавлять категории по умолчанию может только администратор.

```
INSERT INTO category (name, type) VALUES
    ('Покупка товаров', 'EXPENSE'),
    ('Услуги', 'EXPENSE'),
    ('Налоги', 'EXPENSE'),
    ('Зарплата', 'INCOME'),
    ('Подарок', 'INCOME'),
    ('Инвестиции', 'INCOME');
```

	id	name	type	person_id
1	1	Покупка товаров	EXPENSE	<null>
2	2	Услуги	EXPENSE	<null>
3	3	Налоги	EXPENSE	<null>
4	4	Зарплата	INCOME	<null>
5	5	Подарок	INCOME	<null>
6	6	Инвестиции	INCOME	<null>

Добавление пользовательских категорий

- Добавим в таблицу несколько пользовательских категорий.

```
INSERT INTO category (name, type, person_id) VALUES
    ('Трейдинг', 'EXPENSE', 1), -- Категоря EXPENSE для пользователя с id=1.
    ('Трейдинг', 'INCOME', 1), -- Категоря INCOME для пользователя с id=1.
    ('Лотерея', 'INCOME', 1); -- Категоря INCOME для пользователя с id=1.
```

	id	name	type	person_id
1	1	Покупка товаров	EXPENSE	<null>
2	2	Услуги	EXPENSE	<null>
3	3	Налоги	EXPENSE	<null>
4	4	Зарплата	INCOME	<null>
5	5	Подарок	INCOME	<null>
6	6	Инвестиции	INCOME	<null>
7	7	Трейдинг	EXPENSE	1
8	8	Трейдинг	INCOME	1
9	9	Лотерея	INCOME	1

- Скрипт, выполняемый при создании новой пользовательской категории:

```
INSERT INTO category (name, type, person_id) VALUES (?, ?, ?);
```

Получение всех доступных пользователю категорий

```
SELECT * FROM category WHERE category.person_id = ? OR category.person_id IS
NULL;
```

Получение категорий пользователя по подстроке в названии

```
SELECT * FROM category
WHERE (category.person_id = ? OR category.person_id IS NULL)
  AND category.name LIKE '%substr%';
```

Получение категорий пользователя по типу

- Получение всех категорий с типом `INCOME`, доступных пользователю

  ```
  SELECT * FROM category
  WHERE (category.person_id = ? OR category.person_id IS NULL)
    AND category.type LIKE 'INCOME';
  ```

- Получение всех категорий с типом `EXPENSE`, доступных пользователю

  ```
  SELECT * FROM category
  WHERE (category.person_id = ? OR category.person_id IS NULL)
    AND category.type LIKE 'EXPENSE';
  ```

Добавление операций по кошельку

- Процедура, которая осуществляет создание операции для кошелька.
  Происходит проверка, что после добавления операции по кошельку его баланс не меньше нуля и не превышает лимита кошелька. При успешном прохождении проверки баланс в кошельке обновляется, а операция добавляется в таблицу.

```
CREATE OR REPLACE PROCEDURE add_new_operation(
    operation_dttm timestamp,
    amount numeric(10, 2),
    wallet_id IN wallet.id%TYPE,
    category_id IN category.id%TYPE)
    LANGUAGE plpgsql AS
$$
DECLARE
    max_limit     numeric(10, 2);
    balance       numeric(10, 2);
    category_type varchar(7);
BEGIN
```

```
    SELECT w.max_limit, w.balance FROM wallet w WHERE w.id = wallet_id INTO
max_limit, balance;
    SELECT c.type FROM category c WHERE c.id = category_id INTO category_type;

    IF (category_type LIKE 'EXPENSE') THEN
        IF (amount > balance) THEN
            RAISE EXCEPTION 'Expense amount can''t be grater than wallet
balance';
        END IF;
        UPDATE wallet w SET balance = w.balance - amount WHERE w.id = wallet_id;
    ELSE
        IF (category_type LIKE 'INCOME') THEN
            IF (amount + balance > max_limit) THEN
                RAISE EXCEPTION 'Income amount + balance can''t be grater than
wallet max_limit';
            END IF;
            UPDATE wallet w SET balance = w.balance + amount WHERE w.id =
wallet_id;
        END IF;
    END IF;

    INSERT INTO operation (operation_dttm, amount, wallet_id, category_id)
    VALUES (operation_dttm, amount, wallet_id, category_id);
END
$$;
```

- Для добавления пользователем операции должен запускаться данный скрипт:

```
CALL add_new_operation(?, ?, ?, ?);
```

- Например, запустим скрипт со следующими значениями. Баланс кошелька должен уменьшится на 100 (был 1000, должен стать 900), в таблице `operation` должна появиться запись.

```
CALL add_new_operation(TIMESTAMP '2022-12-14 00:33:55', 100, 3, 1);
```

	id	name	max_limit	balance	person_id	currency_id
1	1	Кошелёк 1	1000000.00	0.00	1	1
2	2	Кошелёк 2	10000.00	0.00	1	2
3	4	Кошелёк 1	100.00	30.00	2	3
4	3	Кошелёк 3	10000.00	900.00	1	1

	id	operation_dttm	amount	wallet_id	category_id
1	1	2022-12-14 00:33:55.000000	100.00	3	1

Удаление операции из кошелька

- Процедура, которая удаляет операцию и возвращает изменения суммы, которые были сделаны данной операцией по кошельку.

```
CREATE OR REPLACE PROCEDURE delete_operation(
    operation_id IN operation.id%TYPE)
    LANGUAGE plpgsql AS
$$
DECLARE
    amount        numeric(10, 2);
    category_id   integer;
    category_type varchar(7);
    wallet_id     integer;
BEGIN
    SELECT o.amount, o.category_id, o.wallet_id
    FROM operation o
    WHERE o.id = operation_id
    INTO amount, category_id, wallet_id;

    SELECT c.type FROM category c WHERE c.id = category_id INTO category_type;

    IF (category_type LIKE 'EXPENSE') THEN
        UPDATE wallet w
        SET balance = w.balance + amount
        WHERE w.id = wallet_id;
    ELSE
        IF (category_type LIKE 'INCOME') THEN
            UPDATE wallet w
            SET balance = w.balance - amount
            WHERE w.id = wallet_id;
        END IF;
    END IF;

    DELETE FROM operation o WHERE o.id = operation_id;
END
$$;
```

- Для удаления операции должен запускаться данный скрипт:

```
CALL delete_operation(?);
```

- Например, запустим следующий скрипт. Баланс кошелька должен стать 1000 (был 900, удаляем операцию расхода с суммой 100), в таблице `operation` должна удалиться запись.

```
CALL delete_operation(1);
```

	id	name	max_limit	balance	person_id	currency_id
1	1	Кошелёк 1	1000000.00	0.00	1	1
2	2	Кошелёк 2	10000.00	0.00	1	2
3	4	Кошелёк 1	100.00	30.00	2	3
4	3	Кошелёк 3	10000.00	1000.00	1	1

id	operation_dttm	amount	wallet_id	category_id

Получение общей суммы расходов и доходов по кошельку за весь период

- Сначала добавим несколько операций по кошельку в БД.

```
CALL add_new_operation(TIMESTAMP '2022-12-14 09:33:55', 100, 3, 1);
CALL add_new_operation(TIMESTAMP '2022-11-14 08:00:08', 3000, 3, 5);
CALL add_new_operation(TIMESTAMP '2022-12-14 21:20:50', 500, 3, 7);
CALL add_new_operation(TIMESTAMP '2022-10-14 10:14:37', 1000, 3, 6);
CALL add_new_operation(TIMESTAMP '2022-09-14 06:00:00', 600, 3, 2);
```

	id	operation_dttm	amount	wallet_id	category_id
1	3	2022-12-14 09:33:55.000000	100.00	3	1
2	4	2022-11-14 08:00:08.000000	3000.00	3	5
3	5	2022-12-14 21:20:50.000000	500.00	3	7
4	6	2022-10-14 10:14:37.000000	1000.00	3	6
5	7	2022-09-14 06:00:00.000000	600.00	3	2

- Скрипт для получения общей суммы расходов и доходов по кошельку за весь период:

```
SELECT income, expense
FROM (SELECT SUM(amount) FROM operation
      JOIN category c ON c.id = operation.category_id
      WHERE wallet_id = ? AND c.type LIKE 'INCOME') AS income,
     (SELECT SUM(amount) FROM operation
      JOIN category c ON c.id = operation.category_id
      WHERE wallet_id = ? AND c.type LIKE 'EXPENSE') AS expense
```

	income	expense
1	(4000.00)	(1200.00)

Получение общей суммы расходов и доходов по кошельку за последний месяц

- Скрипт для получения общей суммы расходов и доходов по кошельку за последний месяц:

```
SELECT income, expense
FROM (SELECT SUM(amount)
      FROM operation
      JOIN category c ON c.id = operation.category_id
      WHERE wallet_id = ?
        AND c.type LIKE 'INCOME'
        AND operation_dttm BETWEEN NOW() - INTERVAL '1' MONTH AND NOW()) AS
income,
     (SELECT SUM(amount)
      FROM operation
      JOIN category c ON c.id = operation.category_id
      WHERE wallet_id = ?
        AND c.type LIKE 'EXPENSE'
        AND operation_dttm BETWEEN NOW() - INTERVAL '1' MONTH AND NOW()) AS
expense;
```